***<u>FRONTEND DOCUMENTATION :::</u>***

***SKELETON:***

```
frontend/
│
├── public/
│   ├── index.html
│   ├── favicon.ico
│   └── assets/
│   ├── images/
│   │   └── centurion_logo.png
│   └── icons/
│
├── src/
│
│   ├── components/
│   │   ├── LoginPage.jsx
│   │   ├── PermissionsPage.jsx
│   │   ├── Dashboard.jsx
│   │   ├── LiveFeed.jsx
│   │   └── IoTSimulation.jsx
│
│   ├── api/
│   │   └── backendAPI.js
│
│   ├── styles/
│   │   ├── main.css
│   │   └── animations.css
│
│   ├── utils/
│   │   └── webcam_utils.js
│
│   └── App.jsx
│
├── package.json
└── tailwind.config.js
```

---

*Version: 2.0*
*Project: Centurion AI Security Platform*
*Architecture Type: Secure Biometric Monitoring Interface*
*Frontend Stack: React + TailwindCSS + WebRTC + Chart Visualization Libraries*

---

***FEATURES && SPECIFICATIONS :***

- Authenticate users via facial recognition
  - • Monitor security events in real time

- • Provide administrative analytics
- • Log and visualize unauthorized access attempts
- • Simulate real-world IoT security responses

---

This frontend acts as the **control center for the security system**, communicating with backend services responsible for biometric analysis, logging, alerting, and device control.

---

***FRONTEND FEATURES :***
*It will contain 3 main pages :*

- *Login page*
- *Dashboard page*
- *Simulation page*

---

***SYSTEM FLOW :***

*User Opens Application*
↓
*Login Page Loads*
↓
*Camera Permission Requested*
↓
*User Chooses Authentication Method*
• *Face Login*
• *Face Registration*
• *Email OTP Login*
↓
*Backend Verification*
↓
*If Success → Dashboard*
*If Failure → Security Event Logged*
↓
*Dashboard Displays Metrics and Logs*
↓
*Optional Simulation Page Demonstrates IoT Response*

---

*The frontend is divided into the following layers:*

*1. Presentation Layer (UI Components)*
*2. Application Logic Layer*
*3. API Communication Layer*
*4. Utility Layer*
*5. Styling and Animation Layer*

---

***LOGIN PAGE :***

***Responsibilities:***
***• Handle user authentication***
***• Manage camera access***

- ***Perform face registration***
- ***Send login requests to backend***
- ***Provide OTP email login***

- *User must enable camera access*
- *It should be well designed with modern looking features and sections*
- *Before proceeding to face registration a list of terms and conditions should be provided.The user must read these terms and they should allow user to proceed after a countdown of 15 seconds to allow time for reading the terms and only after this should the register face button be enabled.*
- *Once camera is enabled there should be duo features : register face or login with face*
- *Registered faces must be stored in the database for future remembrance and non-registered faces must be denied access.Once access is denied, the system must proceed to capture the following features and store them in the dashboard page :*

1. *Multiple camera snapshots of person's face*
2. *Face details like age, sex, ethnicity,mood,eyecolor,skin tone,facial features[beards,pimples etc] and overall face shape must be captured.*
3. *Any background or third party faces must be captured if possible plus the facial details and estimate how many background faces are available, if none return null no of faces.*
4. *Logs of time accessed no of attempts of login must also be captured and stored in the console in the dashboard*
5. *An email should be sent to the admin upon unssucessfull login attempts also with a summary of these details*

***FACE AUTHENTICATION WORKFLOW:***

1. *Camera initializes using WebRTC*
2. *Frame captured from webcam*
3. *Face detection performed*
4. *Face features extracted*
5. *Features sent to backend*
6. *Backend compares embeddings*
7. *Authentication decision returned*

- *A succesfully registered users face must have a username loginn for easier database storage and memory so a field to provide username must be provided during the face registration event.*
- *Besides face registration, there should be a separate section where a user provides their email [first time users] and an otp for verification is sent to their email and after entering the code they should complete a provided recapcha to prevent bot access after which is successful, should they poceed to the dashboard by clicking the proceed button.*
- *This page should have a footer labeled "centurion all rights reserved"*

***DASHBOARD PAGE :***

- *Should have a moder designn with the following sections :*

1. *A metric analysis section showing factors like no of attempts , no of successful attempts, no of unsuccessfull attempts, no of registered users[admins],and add admins button which should lead back to login page,a logout button.*
2. *A console like window where real time logs are stored which had earlier talked about incase of unsucessfull login.these logs should be immutable which will be enhanced by blockchain technology later.This console window/terminal should have a start button which when called shold be showing a hacker like vibe showing things like startting server… and upon successful start it should show server online and print an ascii text "CENTURION" before proceeding to show all these logs which were stored in the database.If no logs are available it should print "no recent attempts monitored"*
3. *after the console window all these logs plus data from the metric section should be displayed as aesthetic graphs for easier monitoring and analysis lets say multiple graphs in real time matching real data.*
4. *After the graphs section there should be a snapshots section where snapshots from unsucessfull login are stored and also screenshots that were taken during this unsucessfull events*
5. *There should be a provided immutable summary sectin after this summarising all the events in text form and there should be a send summary to email button which sends it to a registered users email.*
6. *There should be a footer section with a contact other admins section such that when an email and a topic is entered it can be sent to other registered admins a s a communication channel and should also show various social media platforms such as github instagram and linkedin and whatsapp*

7. *There should be a message saying press escape to exit this window such that when esc key is pressed a user is taken back to login page*

- *Aesthetics ,User Experience and User Interface:*

1. *There should be a floating a chat button that when pressed it opens an ai chat window.An api key section will be provided in the code*
2. *An acessability floating button with the following functional features drawer :*

- *a language selection section hence a user can switch between the following languages for the interface :English(defaiult), French, German, Swahili and japanese*
- *A toggle switch for light or dark theme*
- *A read aloud button which should read interface text aloud*
- *A capture screenshot button that captures interface screenshot*

*3.The interface should generally have random dynamic tech doodles in the background and animation like datacenter web like structures.*
*4.The Top container should begin with a capital C with a caligrapthy like font at the beginning which will act as a logo and it should also be positioned at the top of the conact us section..This logo should e rounded*
*5.If any inactivity is detected in the dashboard eg no mouse movements it should auto revert to the login page after 4 mintues for security purposes*
*6.There should be a test/simulation button which should lead to a simulation page which should be on a lockdown or separate browser window for juxtapositioning*

***SIMULATION PAGE ::***

- *Should be providing realtime simulation software which should be matchng login events . This is aimed to provide a realworld demonstaration by using something like a door or led lights.*
- *When one enters the simulation page, it should send the previous browser window which was originally in the dashboard page to the login page.*
- *When one tries to login unsucessully, the door in the simulation page should not open and display an access denied error message or if its ledlights, it should turn red showing acess denied*
- *When one logs in successfully the door should open or ledlights should turn greento show access granted and display an access granted message and the door should lock itself again after this*

- ***IMPORTANT:*** *This simulation should be realtime*

*END.*